

Register map

All registers are 32 bits wide. Offsets are bytes from the controller's base address:

- **Way 2 (ARM):** base = `0x43C0_0000`, set in `vivado/build_ps_system.tcl`. You can see it in the block design's Address Editor.
- **Way 1 (FPGA only):** `ssc_cmd_fsm` uses the offsets directly.

Access types: RO = read only, WO = write only, RW = read/write, W1C = write 1 to clear, W1 = bit is a one-shot command (reads back as 0).

Global

Offset	Name	Access	Bits
0x000	ID	RO	0x53534301 ("SSC", version 1). Read this first to check the bitstream is loaded.
0x004	INT_STATUS	R / W1C	Event flags, see the table below. Write 1 to a bit to clear it.
0x008	INT_ENABLE	RW	A 1 here lets the matching flag drive the IRQ line.
0x00C	BRIDGE_CTRL	RW	[1:0] ch0 source, [3:2] ch0 destination, [5:4] ch1 source, [7:6] ch1 destination. Codes: 0 off, 1 SPI, 2 I2C, 3 UART (the base paper's COSE codes).
0x010	BRIDGE_CNT	RO / write clears	[15:0] bytes moved by ch0, [31:16] by ch1.
0x014	SCRATCH	RW	A free register for bus tests.

INT_STATUS / INT_ENABLE bits

Bit	Name	Kind	Meaning
0	UART_RX	level	UART RX FIFO is not empty
1	UART_TX_IDLE	level	UART TX FIFO is empty and the line is idle
2	UART_PARITY	event	Parity error (the byte is still stored)
3	UART_FRAME	event	Framing error: the stop bit was 0 (byte dropped)
4	UART_BREAK	event	Line held low for a whole frame
5	UART_RX_OVF	event	A byte arrived while the RX FIFO was full
6	SPI_DONE	event	Master: the queue ran empty. Slave: CS# went high
7	SPI_RX	level	SPI RX FIFO is not empty
8	SPI_RX_OVF	event	A word arrived while the RX FIFO was full
9	I2C_DONE	event	Command finished (also after a NACK or lost arbitration)
10	I2C_NACK	event	The slave did not acknowledge
11	I2C_ARB_LOST	event	Another master won the bus
12	I2C_RX	level	I2C RX FIFO is not empty
13	UART_TX_OVF	event	Write to a full UART TX FIFO
14	SPI_TX_OVF	event	Write to a full SPI TX FIFO
15	I2C_TX_OVF	event	Write to a full I2C TX FIFO

A **level** flag sets itself again straight after you clear it, for as long as its condition stays true. That is why the ISR empties the UART RX FIFO before it clears bit 0.

Status register layout (UART, SPI and I2C use the same layout)

Bits	Meaning
0	TX_EMPTY
1	TX_FULL
2	RX_EMPTY
3	RX_FULL
4	BUSY (UART: transmitter busy)
12:8	TX_COUNT (0..16)
20:16	RX_COUNT (0..16)

UART (0x100)

Offset	Name	Access	Bits
0x100	UART_CTRL	RW	[0] TX_EN, [1] RX_EN, [3:2] data bits (0=5, 1=6, 2=7, 3=8), [4] PAR_EN, [5] PAR_ODD, [6] STOP2, [7] FLOW (RTS/CTS), [8] LOOPBACK. W1: [16] TX_FLUSH, [17] RX_FLUSH
0x104	UART_BAUD	RW	[15:0] whole part of the divisor, [19:16] fraction in 1/16. The divisor is $f_{\text{clk}} / (16 \times \text{baud})$. Reset value $0x00040036 = 54 + 4/16 = 115200$ baud.
0x108	UART_STATUS	RO	Common layout, plus [5] RX_BUSY and [6] CTS_OK
0x10C	UART_TXDATA	WO	[7:0] goes into the TX FIFO
0x110	UART_RXDATA	RO	[7:0] byte taken from the RX FIFO. [31] EMPTY: there was no byte, so nothing was taken

To compute UART_BAUD in software, work out $\text{div16} = \text{round}(100 \text{ MHz} / \text{baud})$, then write $((\text{div16} \& 15) \ll 16) \mid (\text{div16} \gg 4)$.

SPI (0x200)

Offset	Name	Access	Bits
0x200	SPI_CTRL	RW	[0] EN, [1] CPOL, [2] CPHA, [3] LSB_FIRST, [8:4] word length – 1 (3..31, for 4..32-bit words), [10:9] CS_SEL, [11] CS_MANUAL, [12] CS_LEVEL (1 = CS# low), [13] SLAVE, [14] LOOPBACK. W1: [16] TX_FLUSH, [17] RX_FLUSH, [18] ABORT
0x204	SPI_CLKDIV	RW	$\text{SCLK} = 100 \text{ MHz} / (2 \times (\text{DIV} + 1))$. The minimum is 3 (12.5 MHz). Reset value 49 gives 1 MHz.
0x208	SPI_STATUS	RO	Common layout
0x20C	SPI_TXDATA	WO	The word goes into the TX FIFO. In master mode, a word in the FIFO starts a transfer.
0x210	SPI_RXDATA	RO	The word taken from the RX FIFO. Reads 0 when the FIFO is empty, so check RX_EMPTY first.

Chip select works in one of two ways:

- **Automatic** (CS_MANUAL = 0): CS# goes low while words are queued, and goes high when the queue runs empty.
- **Manual** (CS_MANUAL = 1): software holds CS# low with CS_LEVEL. Flash commands use this mode.

I2C (0x300)

Offset	Name	Access	Bits
0x300	I2C_CTRL	RW	[0] EN, [1] AUTO_WR (bridge mode: each TX byte becomes a 1-byte write to ADDR). W1: [16] TX_FLUSH, [17] RX_FLUSH, [18] ABORT
0x304	I2C_PRESCALE	RW	$f_{SCL} \approx 100 \text{ MHz} / (4 \times (P + 1))$. Use 249 for 100 kHz (the reset value) or 62 for 400 kHz mode. At 62 the nominal rate is 397 kHz, and test [18] measured 383 kHz because of the bus rise time.
0x308	I2C_ADDR	RW	[9:0] slave address, [15] TEN_BIT
0x30C	I2C_CMD	WO	[7:0] LEN, [8] READ, [9] STOP, [10] STOP_ONLY. Writing this register starts the transaction.
0x310	I2C_STATUS	RO	Common layout, plus [5] HOLDING (bus kept for a repeated START), [6] NACK, [7] ARB_LOST, [24] BUS_BUSY, [25] SCL level, [26] SDA level
0x314	I2C_TXDATA	WO	[7:0] goes into the TX FIFO
0x318	I2C_RXDATA	RO	[7:0] byte taken from the RX FIFO, [31] EMPTY

Example: the PmodTMP2 read (slide 13)

```
I2C_ADDR    = 0x4B
I2C_TXDATA  = 0x00          pointer = temperature register
I2C_CMD     = 0x001         write 1 byte, no STOP  -> S 96 A 00 A
    wait for INT_STATUS.I2C_DONE
I2C_CMD     = 0x302         read 2 bytes + STOP    -> Sr 97 A 0C A 80 N P
    wait for INT_STATUS.I2C_DONE
read I2C_RXDATA twice      -> 0x0C, 0x80  -> (0x0C80 >> 3) x 0.0625 = 25.0 C
```